



Introduction to NLP

AIAA 4051 Lecture 19

Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Motivation of Scaling laws

- Model sizes have increased dramatically over time, moving from millions of parameters to hundreds of billions and even trillions.

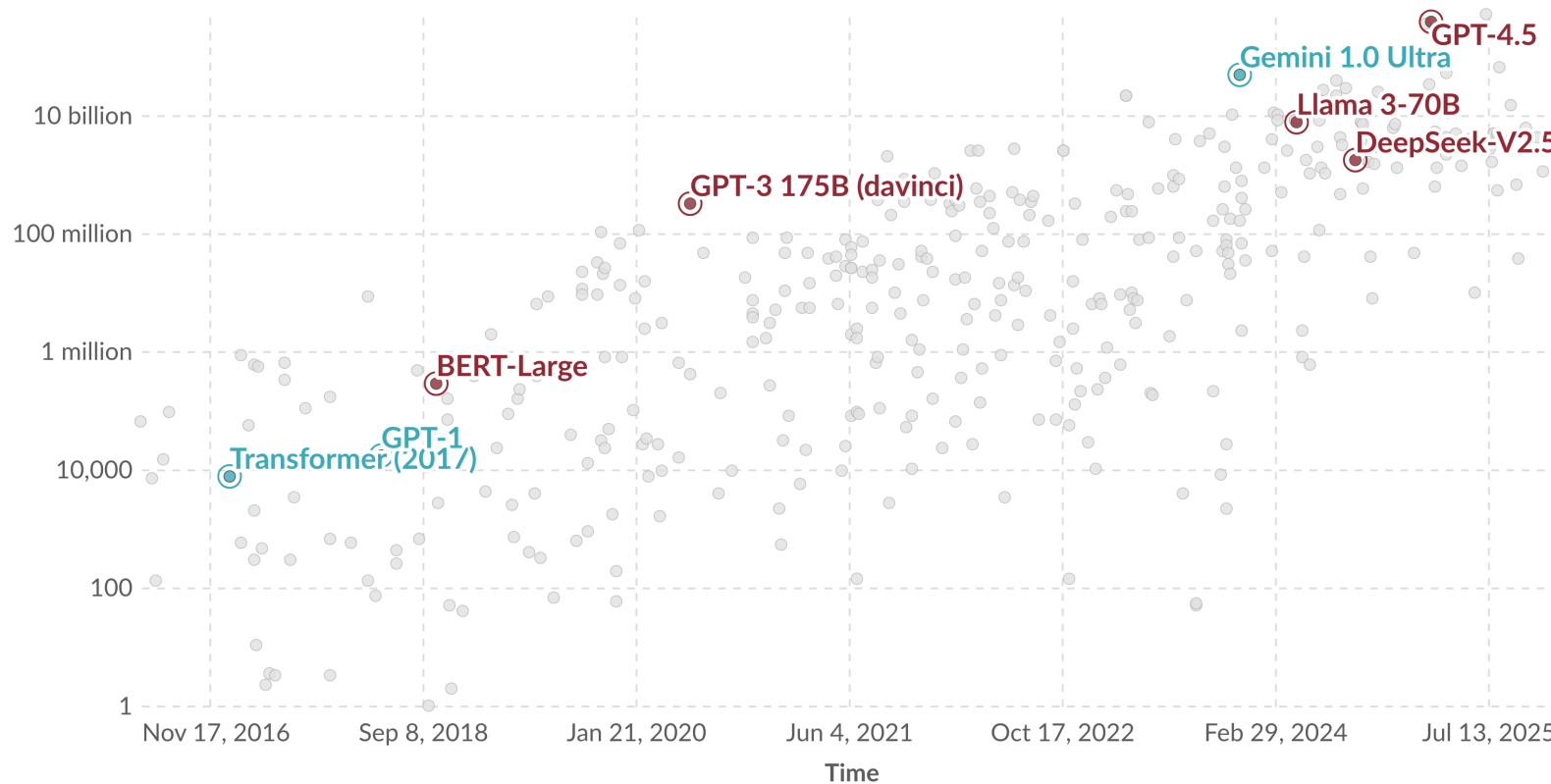
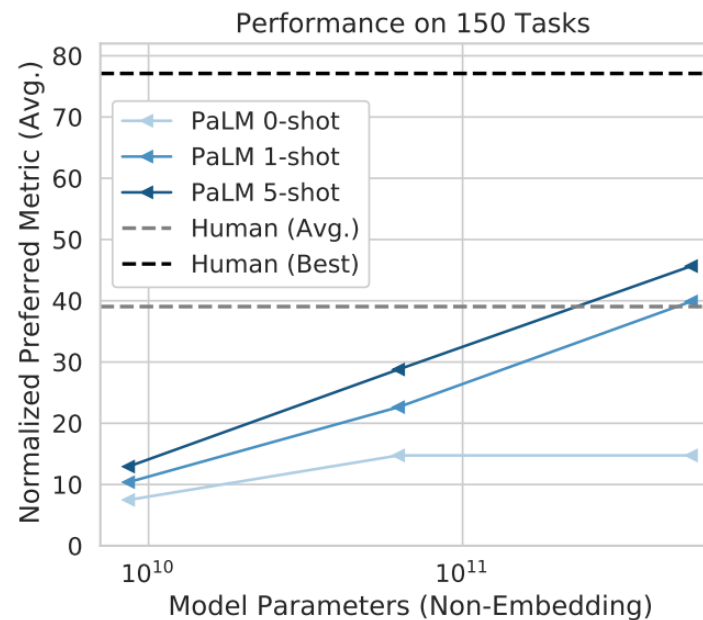
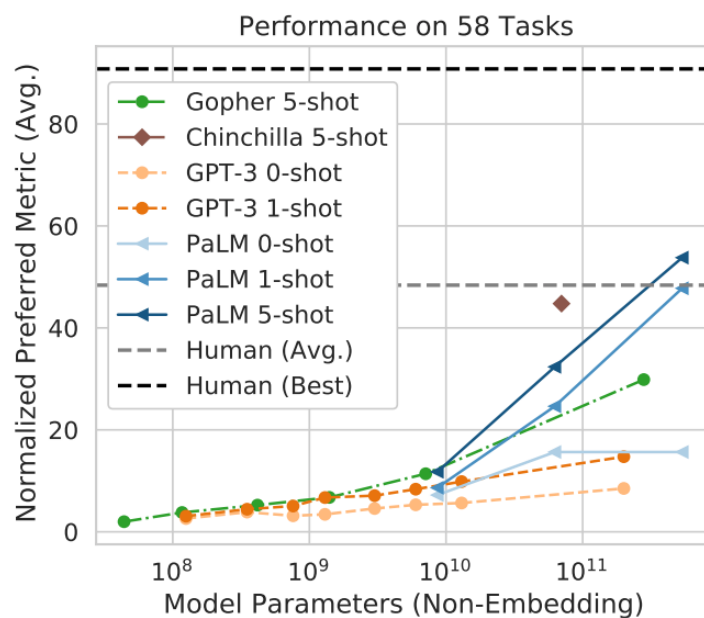


Image source: <https://ourworldindata.org/grapher/exponential-growth-of-parameters-in-notable-ai-systems>

Model Size vs Accuracy

- Larger language models have better zero/few-shot performance.
- Each 10× increase requires much more compute and data.

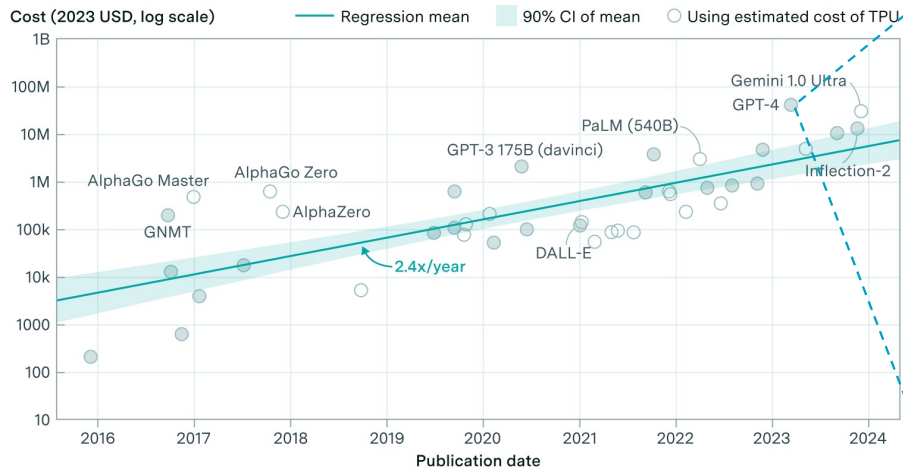


Is the next 10× increase in **scale(?)** worth it, and how much performance improvement will it deliver?

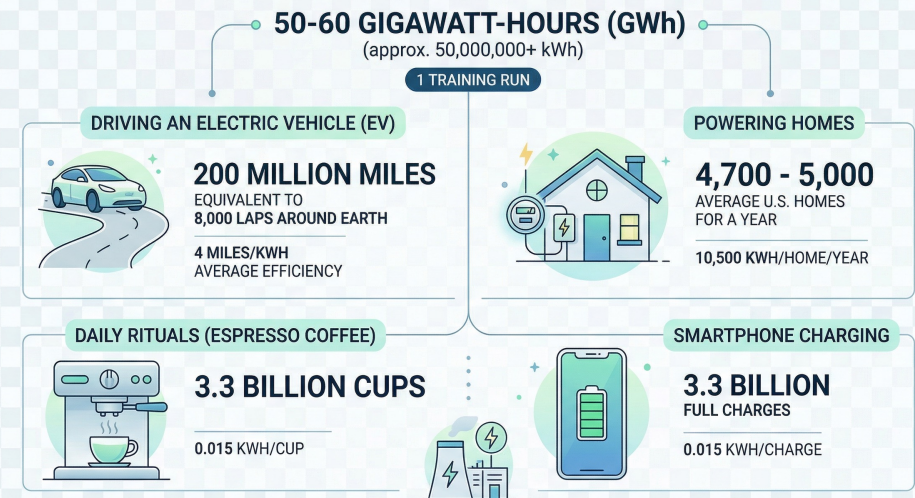
Cottier B, Rahman R, Fattorini L, et al. The rising costs of training frontier AI models[J]. arXiv preprint arXiv:2405.21015, 2024.

Motivation of Scaling laws

- A million-dollar question: How should I invest a fixed amount of resources on **data** or **GPU**?
 - GPT-4 pretraining is estimated to have cost over \$100 million.
 - Decision-makers need to predict what model size, data scale, and compute budget are required to reach a target level of performance.



ESTIMATED ENERGY CONSUMPTION OF TRAINING A LARGE LLM (LIKE GPT-4)



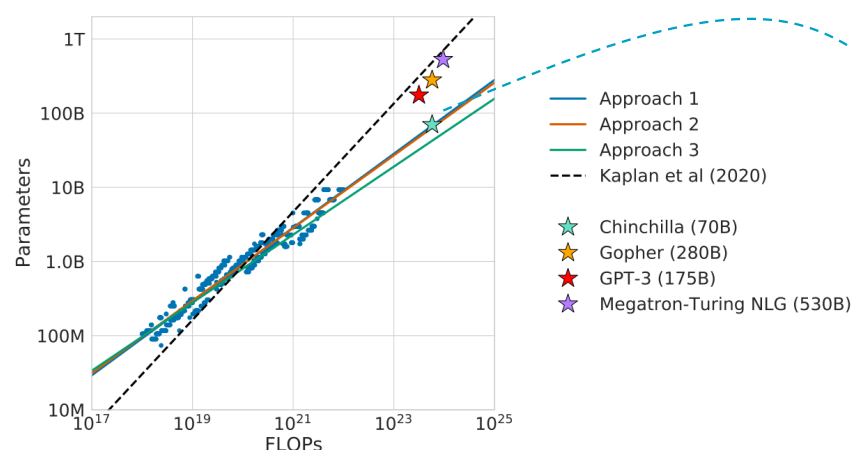
Cottier B, Rahman R, Fattorini L, et al. The rising costs of training frontier AI models[J]. arXiv preprint arXiv:2405.21015, 2024.

| What is Scaling

- “**Scaling**” means larger model size (# of parameter N), more compute (FLOS C) and more data (# of tokens D).
- Increasing only one dimension is often inefficient. Good performance comes from balancing all three.
 - Case 1 (**overfitting**): A $10\times$ larger model with the same amount of data can memorize the training set more easily, but it often gains little in generalization.
 - Case 2 (**underfitting**): With $10\times$ more data but the same small model, there is more information to learn from, but the model may not have enough capacity to absorb it effectively.
 - Case 3 (**right scaling**): When model size, data, and compute all grow together, the model can learn more patterns and achieve more consistent improvement.

Constraints of Real world

- Example of constraints: the total computing budget
 - **FLOPs** provides a unified way to measure that budget.



Random	25.0%
Average human rater	34.5%
GPT-3 5-shot	43.9%
Gopher 5-shot	60.0%
Chinchilla 5-shot	67.6%
Average human expert performance	89.8%
June 2022 Forecast	57.1%
June 2023 Forecast	63.4%

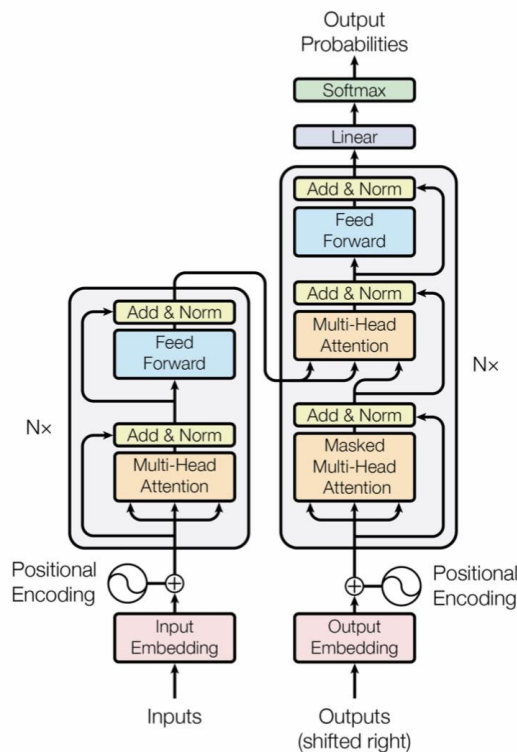
Under the same compute budget, can a smaller model outperform Gopher with 280B parameters?

Use scaling law to make the right decision: Chinchilla (**70B**) with more data (1.4T tokens) and achieved better performance at the same compute.

Hoffmann J, Borgeaud S, Mensch A, et al. Training compute-optimal large language models[J]. arXiv preprint arXiv:2203.15556, 2022, 10.

FLOPs

- Floating point operations per second (FLOPS, flops or flop/s).
- Each FLOP can represent an addition, subtraction, multiplication, or division of floating-point numbers.
- The total FLOP of training a model (e.g., GPT-4) provides a basic approximation of computational costs of training.



Significant cost of training a Transformer comes from matrix-vector products and we focus on FLOPs of such products

Example of matrix-vector products in Transformer:

- Attention query: $W_Q h$
- Projection back: $W_P h$
- Feedforward (ff): $W_{FF} h$

Ignore FLOPs for

- Bias vector addition
- Layer normalization
- Residual connections
- Non-linearities
- Softmax

FLOPs: Matrix-vector Multiplication

- Requires $2mn$ ($2 \times$ matrix size) operations to multiply $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^n$
- **The factor 2:** 1 for multiplication, 1 for addition.

$$\begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} A_{11}x_1 + A_{12}x_2 + \cdots + A_{1n}x_n \\ A_{21}x_1 + A_{22}x_2 + \cdots + A_{2n}x_n \\ \vdots \\ A_{m1}x_1 + A_{m2}x_2 + \cdots + A_{mn}x_n \end{bmatrix}$$

- For multiplying $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, one needs $2mnp$ operations
- This is just for the forward propagation.

FLOPs: Matrix Multiplication

- Backward passes calculate the derivatives of loss with respect to both hidden state (*for further backprop*) and parameters (*for gradient*)

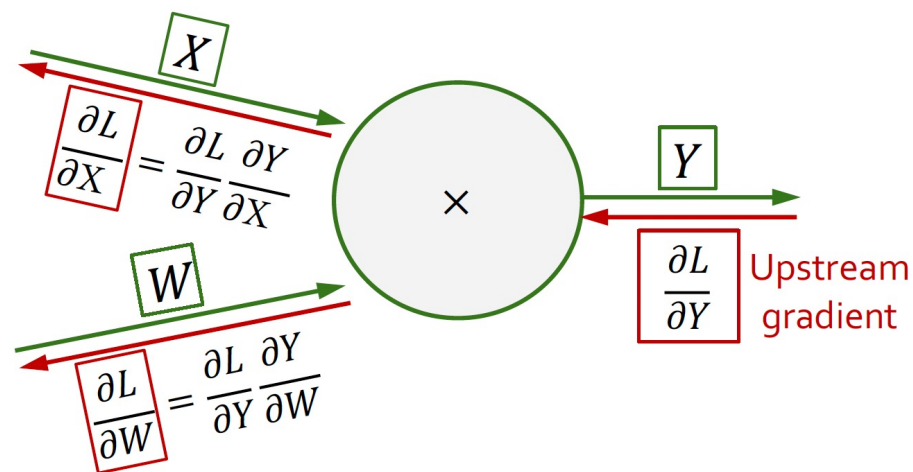
Due to the chain-rule and inner product between W and X , back propagations are also matrix-vector products:

$$\frac{\partial L}{\partial X} = W' \times \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial W} = X' \times \frac{\partial L}{\partial Y}$$

We compute $\frac{\partial L}{\partial X}$ to propagate the gradient to earlier layers

We compute $\frac{\partial L}{\partial W}$ to update model parameters

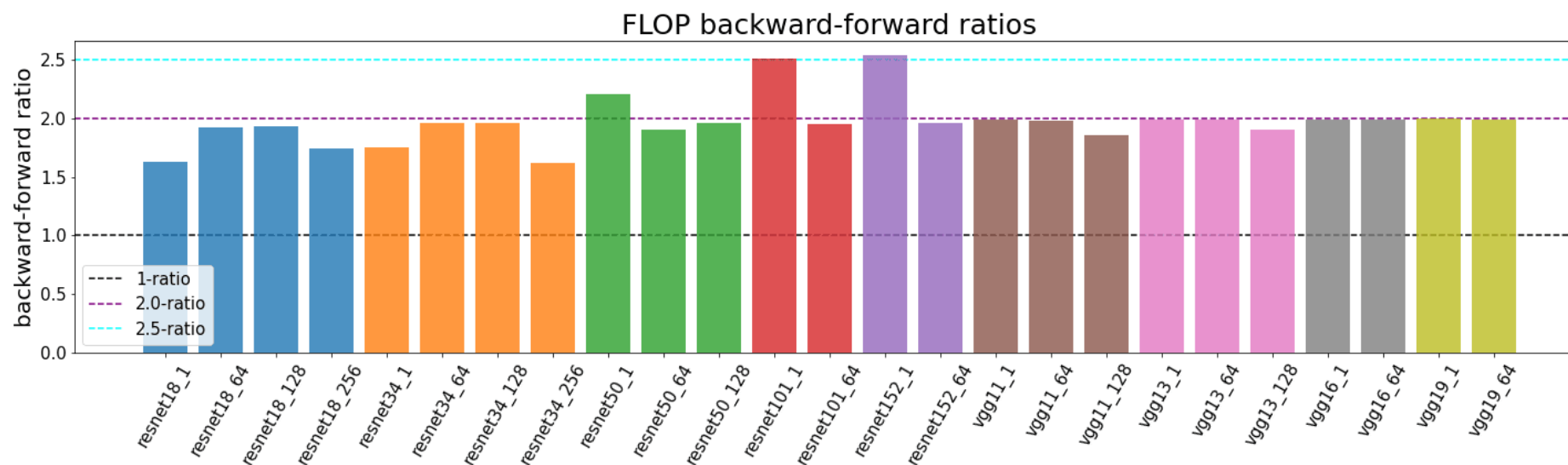


- FLOPs for backward pass is roughly twice of forward pass.

FLOPs: Matrix Multiplication

- FLOPs for backward pass is roughly twice of forward pass.

This ratio depends on various parameters (layers, width-depth ratio, batch size)



Hobbhahn M, Sevilla J. What's the backward-forward flop ratio for neural networks?[J]. Published online at epochai. org, 2021.

FLOPs: Matrix Multiplication

- If $W \in \mathbb{R}^{d_{in} \times d_{out}}$, then the number of parameters for multiplying W with a vector h is $|W| = d_{in} \times d_{out}$.

FLOPs of a single layer with a single matrix-vector product is
 $6 \times (\# \text{ tokens}) \times (\# \text{ of parameters})$

- Example: If $W \in \mathbb{R}^{4096 \times 11008}$, with 8 tokens, the training FLOPs are

$$6 \times B \times |W| = 6 \times 8 \times 4096 \times 11008 \approx 2.16 \times 10^9$$

Transformer FLOPs

- Let N be number of parameters (the sum of sizes of all matrices)
- Let D be the number of tokens in pre-training dataset
- Forward pass:
 - FLOPs for forward pass on a single token is roughly $2N$
 - FLOPs for forward pass for the entire dataset is roughly $2ND$
- Backward pass:
 - FLOPs for backward pass is roughly twice of forward pass
 - FLOPs for backward pass for the entire dataset is roughly $4ND$
- The total cost of pre-training on this dataset is: $C \approx 6ND$

If you have a fixed compute budget C , increasing D means decreasing N

For a detailed calculation of $6ND$, see:

<https://www.lesswrong.com/posts/fnjKpBoWJXcSDwhZk/what-s-the-backward-forward-flop-ratio-for-neural-networks>

Calculating Transformer's N

# parameters	Exact formula	Approximate formula
Multi-head attention	$4(d_{model}^2 + d_{model})$	$4d_{model}^2$
Feed-forward network	$2d_{model}d_{ff} + d_{model} + d_{ff}$	$2d_{model}d_{ff}$
Layer normalization	$2d_{model}$	0
Transformer Encoder	$4d_{model}^2 + 2d_{model}d_{ff} + 9d_{model} + d_{ff}$	$4d_{model}^2 + 2d_{model}d_{ff}$
		or
		$12d_{model}^2 \approx 10d_{model}^2$
Transformer Decoder	$8d_{model}^2 + 2d_{model}d_{ff} + 15d_{model} + d_{ff}$	$8d_{model}^2 + 2d_{model}d_{ff}$
		or
		$16d_{model}^2 \approx 10d_{model}^2$

For a detailed calculation, see

<https://medium.com/data-science/how-to-estimate-the-number-of-parameters-in-transformer-models-ca0f57d8dff0>

Calculating Transformer's N

n_{layer}	d_{model}	Parameters (N)
4	512	13M
6	768	42M
10	1280	197M
16	2048	810M
24	3072	2.7B
40	5120	13B
64	8192	52B

$$N = \underbrace{12 \times n_{\text{layers}} \times d_{\text{model}}^2}_{\text{Non-embedding}} + \underbrace{(n_{\text{vocab}} + n_{\text{pos}}) \times d_{\text{model}}}_{\text{Embedding}}$$

For example:

$$N_{\text{non-embedding}} = 12 \times 64 \times 8192^2 = 51.5B$$

Vocab size = 65536

Positional emb size = 1000

$$N_{\text{embedding}} = (65536 + 1000) \times 8192 = 0.5B$$

Askell A, Bai Y, Chen A, et al. A general language assistant as a laboratory for alignment

Calculating Transformer's C

n_{layer}	d_{model}	Parameters (N)	Training FLOPs
4	512	13M	3.0e19
6	768	42M	1.0e20
10	1280	197M	4.7e20
16	2048	810M	1.9e21
24	3072	2.7B	6.5e21
40	5120	13B	3.0e22
64	8192	52B	1.2e23

Given the pre-training data with 400B tokens

Training cost (FLOPs):

$$\begin{aligned}C &\approx 6ND \\&= 6 \times 52 \times 10^9 \times 400 \times 10^9 \\&= 1.24 \times 10^{23}\end{aligned}$$

Askell A, Bai Y, Chen A, et al. A general language assistant as a laboratory for alignment

Calculating Transformer's C

- Consider HyperCLOVA, an 82B parameter model that was pre-trained on 150B tokens, using a cluster of 1024 A100 GPUs.

- Training cost (FLOPs):

$$\begin{aligned}C &\approx 6ND \\&= 6 \times 82 \times 10^9 \times 150 \times 10^9 = 7.3 \times 10^{22}\end{aligned}$$

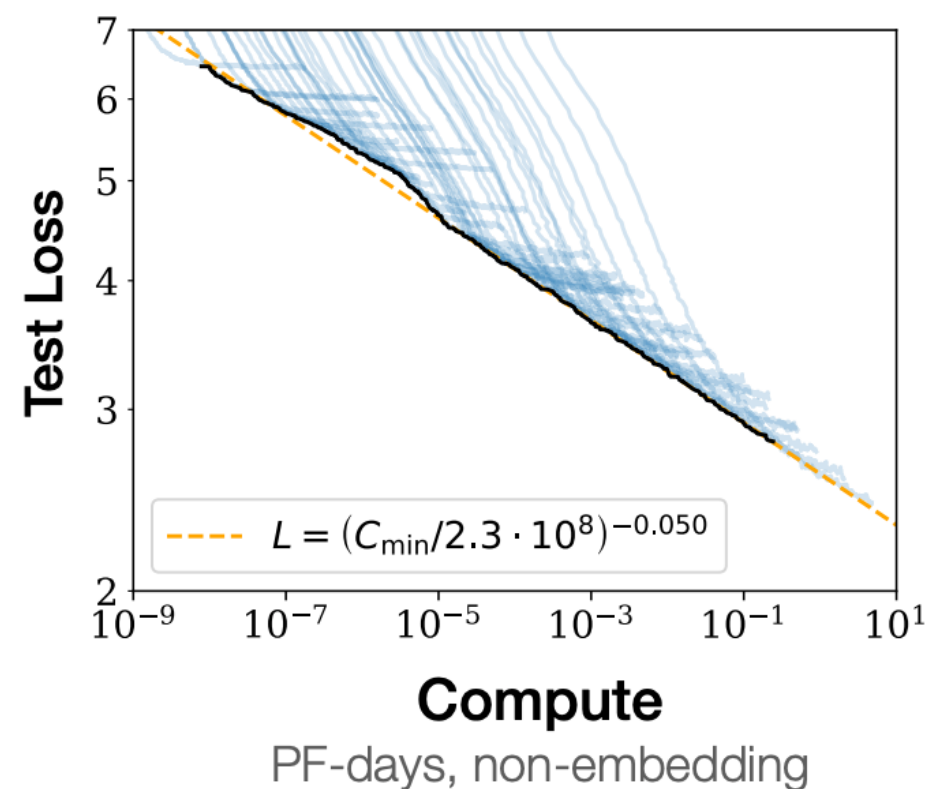
- The peak throughput of A100 GPUs is 312 teraFLOPS or 3.12×10^{14}
- How long would this take? $\text{Duration} = \frac{\text{model compute cost}}{\text{cluster throughput}} = \frac{7.3 \times 10^{22}}{3.12 \times 10^{14} \times 1024} = 2.7 \text{ days}$
- In other words, $C = 2.7 \text{ TF} = 2.7 \times 10^{-3} \text{ PF}$
 - TF=TeraFLOPs, and PF=PeraFLOPs

Calculating Transformer's C

- According to the paper, training took 13.4 days. The estimate is 5 times off, why?
- These estimates can be slightly off in practice:
 - ignore many operations like softmax, Relu/Gelu activations, layer Norm etc.
 - In distributed training, cross-GPU All-Reduce communication can consume 15–30% of training time.
 - Memory bandwidth constraints often limit GPU utilization to 30–60% of the theoretical peak.
 - Cached activations can be swapped in and out between GPU caches and CPU memory
 - *More on KV-caching next lecture.*
 - Additional overheads include checkpointing, data loading, and recovery from node failures or OOM (out-of-memory) exceptions.

Kaplan's scaling laws

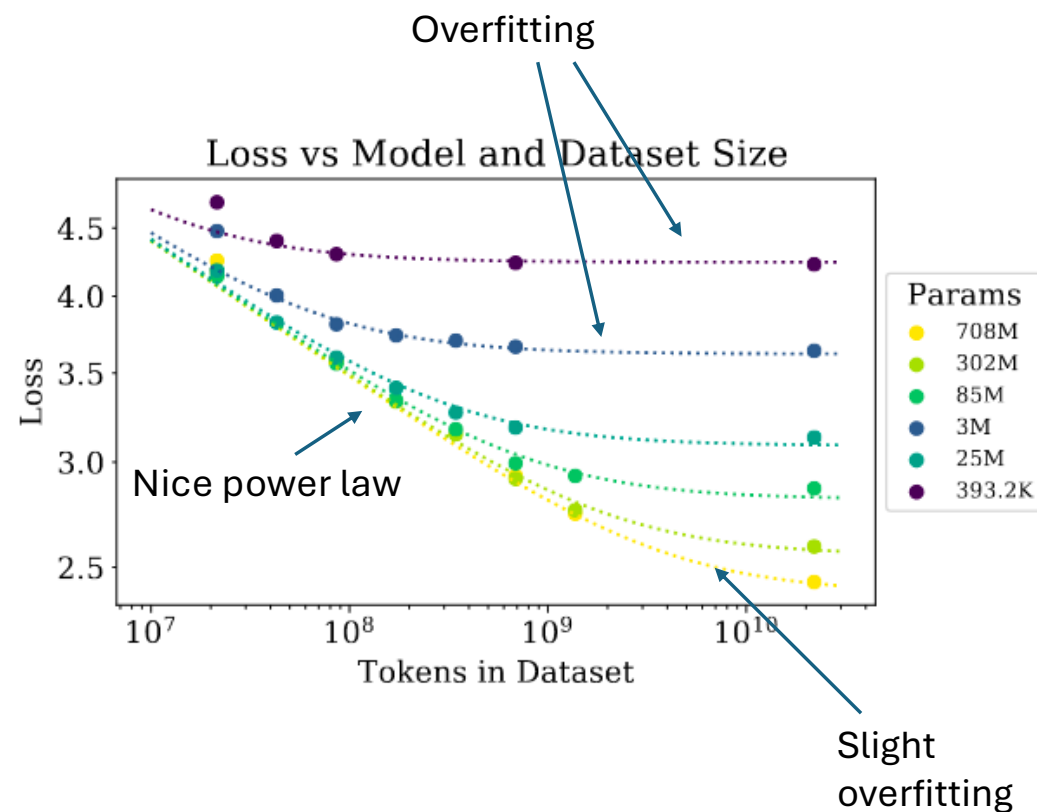
- **Blue curves:** Each blue curve shows the loss of a fixed-size model over a fixed number of training steps.
- **Black points:** The black points mark the lowest achievable loss across models, forming the compute-efficient frontier.
- **Orange dashed line:** a **power-law** fit to the frontier, which is the scaling law.
- Scaling laws show that model performance improves in a predictable way with scale



Note: power-law is used to describe quantities varying across magnitudes.

Kaplan's scaling laws

- The x-axis shows the number of training tokens
- The y-axis shows the loss
- Each curve corresponds to a model of a different size
- Training loss/test loss curves follow predictable power laws
- Larger models have steeper curves, meaning that with the same amount of data, they achieve greater loss reduction => **# of data and parameters should grow simultaneously!**

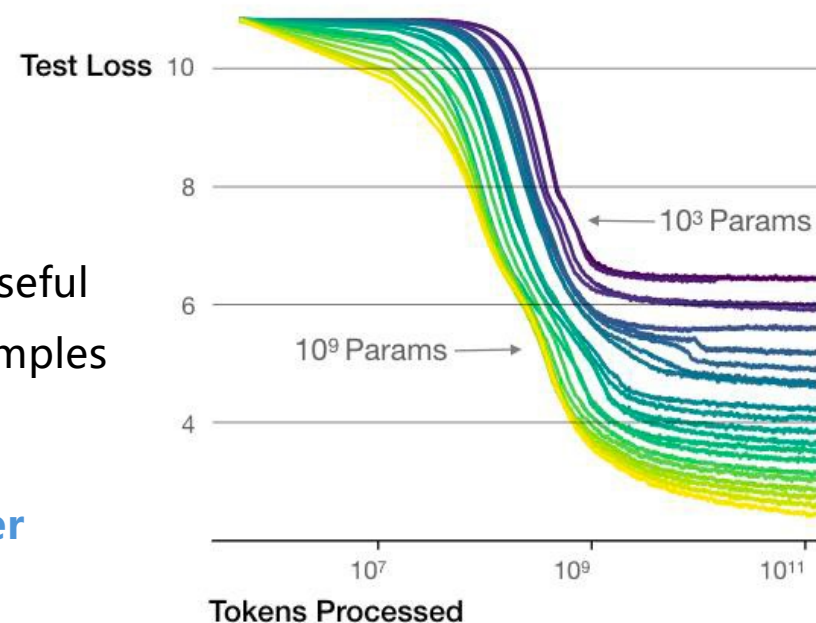


Kaplan J, McCandlish S, Henighan T, et al. Scaling laws for neural language models[J]. arXiv preprint arXiv:2001.08361, 2020.

Kaplan's scaling laws

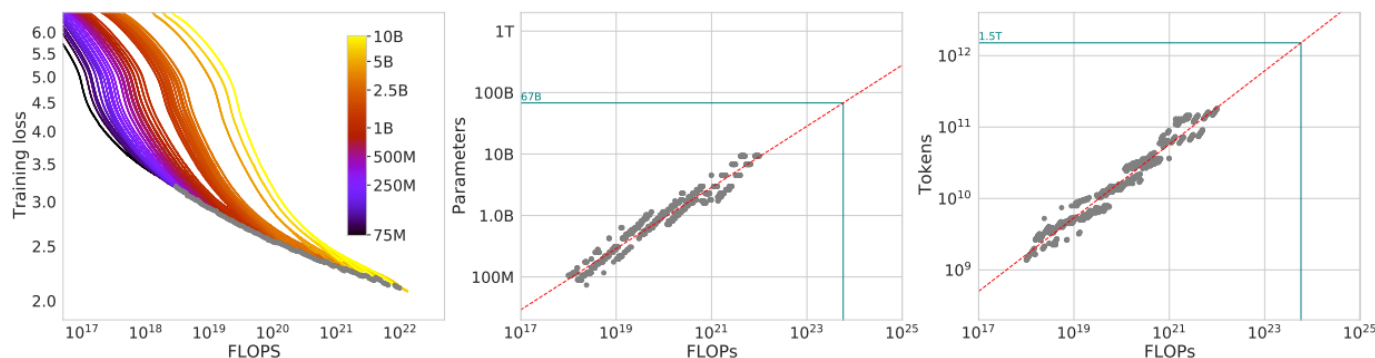
- The x-axis shows the number of tokens processed during training
- The y-axis shows the test loss
- Each curve represents a model with a different number of parameters
- Model size improves data efficiency: bigger models extract useful patterns from data more **effectively**, so they need fewer examples to learn the same thing.
- **The Kaplan scaling law: Bigger models often achieve better final performance.**

Larger models require **fewer samples** to reach the same performance



Kaplan J, McCandlish S, Henighan T, et al. Scaling laws for neural language models[J]. arXiv preprint arXiv:2001.08361, 2020.

Chinchilla's scaling law

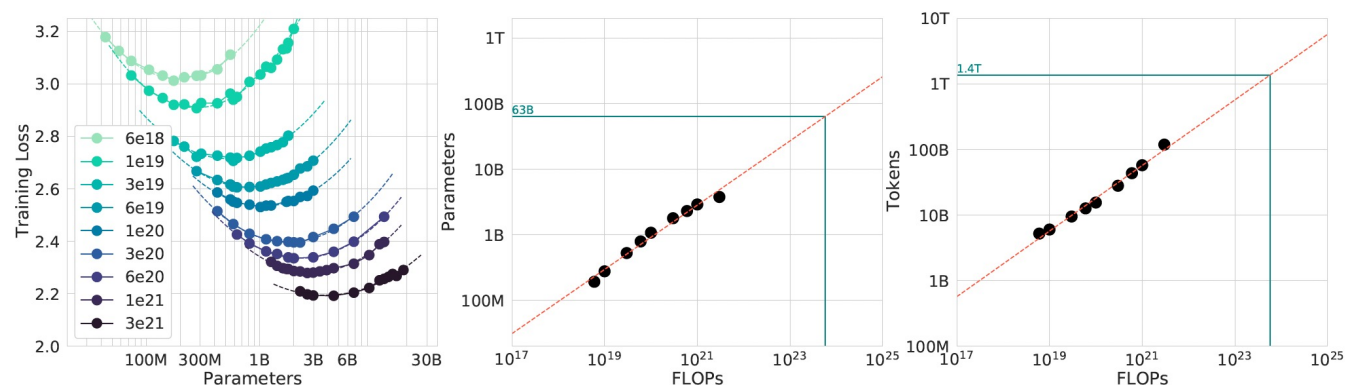


- Training runs for models from 70M to 10B parameters under different compute budget.
 - Different from Kaplan's experiments: larger model size, optimal learning rate schedules.
- Middle: Estimated **optimal** model size for each compute budget.
- Right: Estimated optimal number of training tokens for each compute budget.
- Example: With a compute budget of 6×10^{23} FLOPs (about 1 million A100 GPU hours)
 - Case 1: training a model of roughly 174B parameters with relatively limited data
 - Case 2: a smaller model of about 67B parameters trained on around 1.3T tokens
 - In experiments, case 2 performs better, challenging the earlier Kaplan's scaling law "bigger is always better".

Hoffmann J, Borgeaud S, Mensch A, et al. Training compute-optimal large language models[J]. arXiv preprint arXiv:2203.15556, 2022.

Chinchilla's scaling law

The IsoFLOP curves



- Training runs given different FLOPs C . For each C , vary model size to find the optimal size.
- Middle: Estimated **optimal** model size for each compute budget.
- Right: Estimated optimal number of training tokens for each compute budget.

Hoffmann J, Borgeaud S, Mensch A, et al. Training compute-optimal large language models[J]. arXiv preprint arXiv:2203.15556, 2022.

Chinchilla's scaling law

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

Kaplan under-estimated the utility of model parameters due to:

- 1) smaller model sizes;
- 2) non-optimal learning rates.

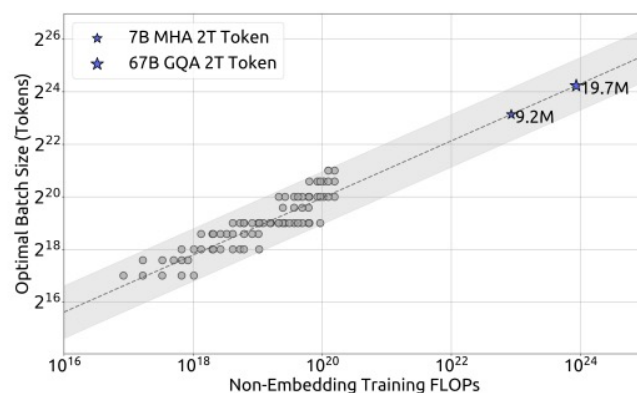
Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29, 968	8.0 Billion
1 Billion	1.21e+20	1/4, 761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Gopher is a larger model with 280B parameters, but was trained on insufficient number of tokens.

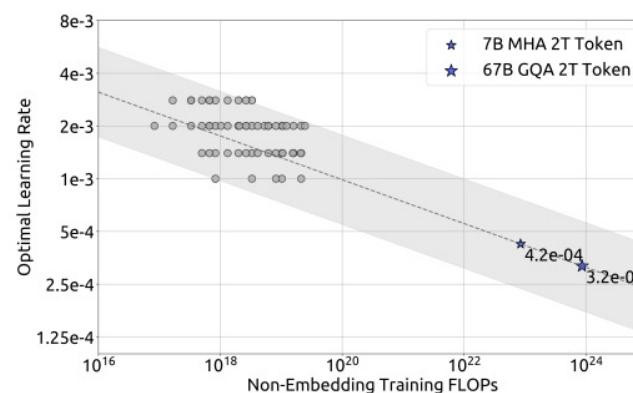
Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

| Kaplan or Chinchilla?

Variants of scaling laws



(a) Batch size scaling curve



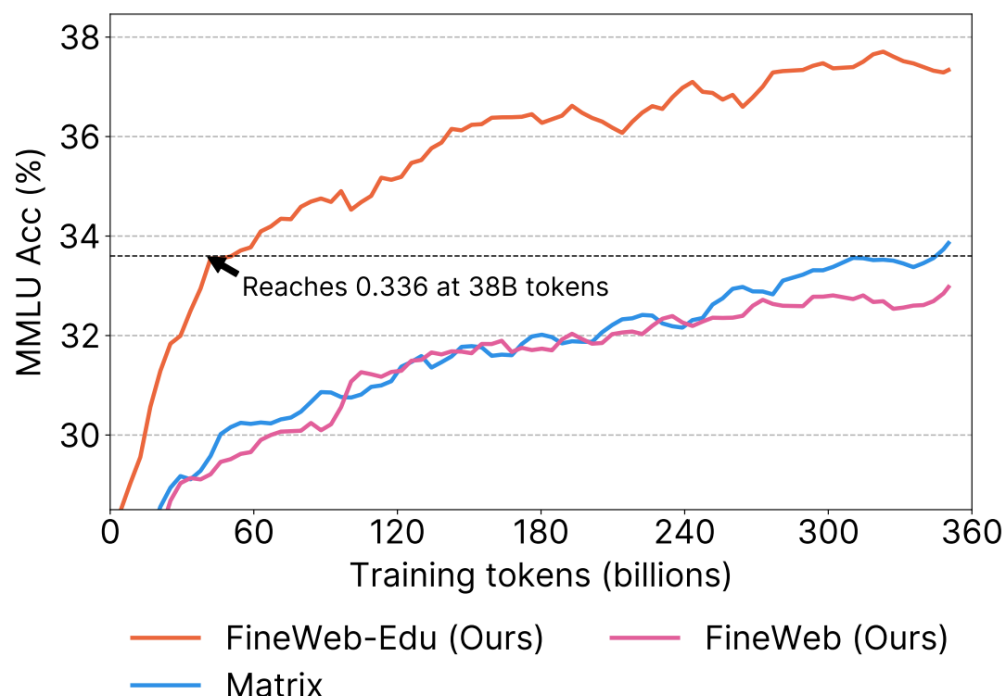
(b) Learning rate scaling curve

- The x-axis is training FLOPs, and the y-axis is the optimal batch size (left) or learning rate (right)
- As compute increases, the optimal batch size grows, while the optimal learning rate decreases
- Fitting these scaling curves on small-scale experiments, one can predict near-optimal hyperparameters for large-scale training and avoid expensive hyperparameter search

Bi X, Chen D, Chen G, et al. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:2401.02954, 2024.

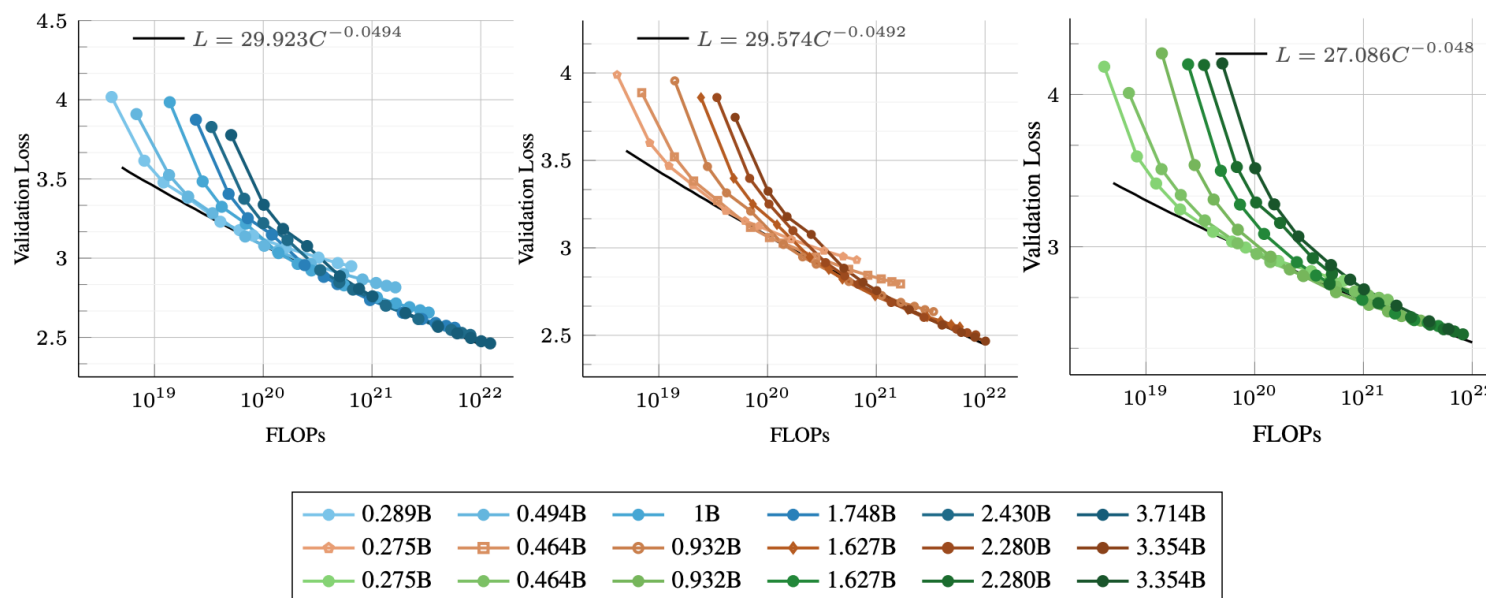
Variants of scaling laws

- The FineWeb-Edu reaches 33.6% accuracy at just 38B **high-quality** training tokens, while the Matrix dataset requires about 300B tokens from a less curated web corpus to reach a similar result.
- Traditional scaling laws treat token count as the main data variable and assume similar token quality. FineWeb shows that high-quality tokens matters more than raw token count.



Penedo G, Kydlíček H, Lozhkov A, et al. The fineweb datasets: Decanting the web for the finest text data at scale[J]. Advances in Neural Information Processing Systems, 2024, 37: 30811-30849.

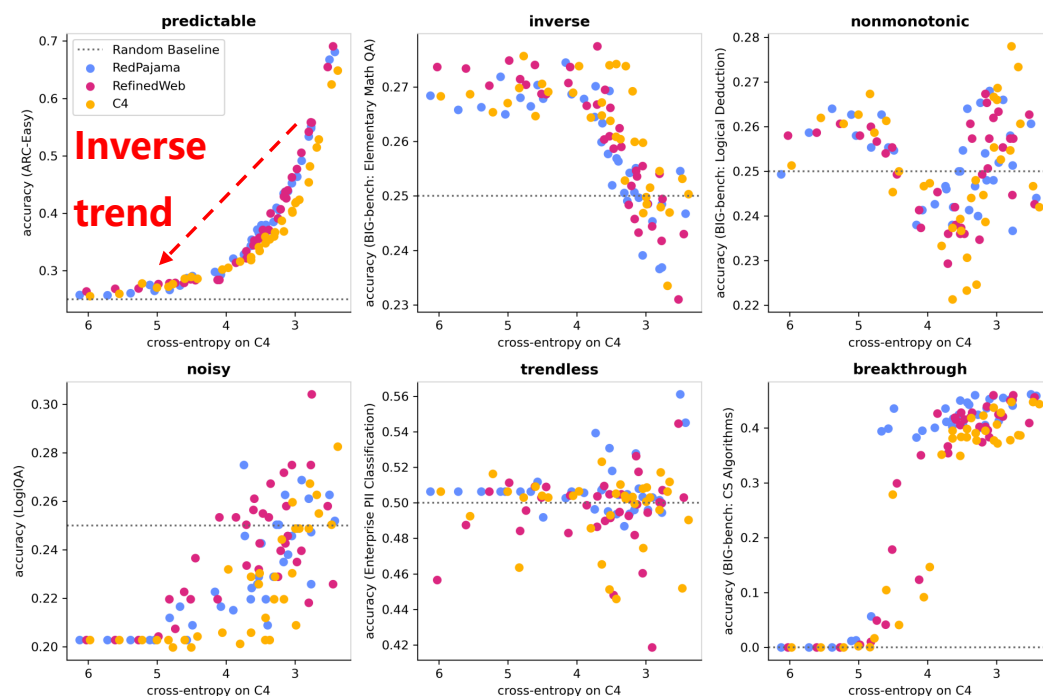
Variants of scaling laws



- This figure shows that scaling laws also hold for **multimodal language models**, not only for text-only LLMs.

Shukor M, Fini E, da Costa V G T, et al. Scaling laws for native multimodal models[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2025: 12-23.

Limitations of Scaling Laws



• Inverse scaling:

- The x-axis is cross-entropy loss, which is often used as a proxy for pretraining quality, and the y-axis is downstream task performance (varied according to task types).
- Pretraining scaling laws do not automatically transfer to downstream **tasks that cannot be measured by cross-entropy loss of next token prediction.**
- A smooth power law in loss does not guarantee a smooth curve in other end-task metrics

Lourie N, Hu M Y, Cho K. Scaling laws are unreliable for downstream tasks: A reality check[J]. arXiv preprint arXiv:2507.00885, 2025.

Why inverse scaling

- preference to repeat memorized sequences over following in-context instructions
- imitation of undesirable patterns in the training data
- tasks containing an easy distractor task which LMs could focus on, rather than the harder real task
- correct but misleading few-shot demonstrations of the task

A demonstration of memorization when LLMs use more training computation.

Model: GPT-4

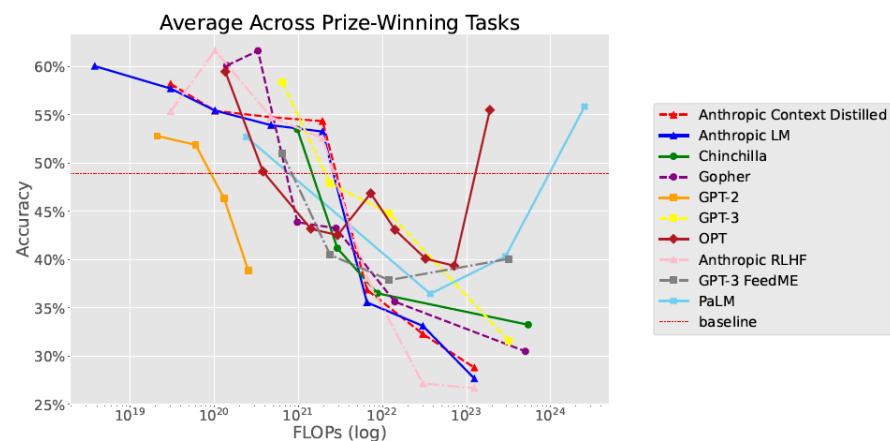
Repeat my sentences verbatim:

Input: Bennet with civil self-command to Miss Lucas.
Output: Bennet with civil self-command to Miss Lucas.

Input: Collins was also in the same state of angry pride.
Output: Collins was also in the same state of angry pride.

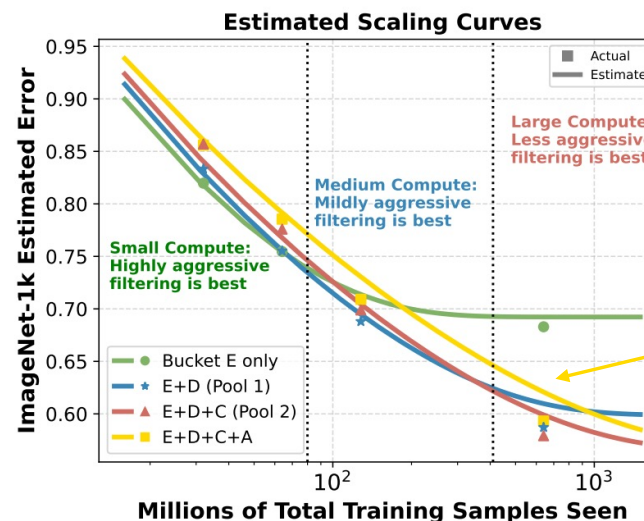
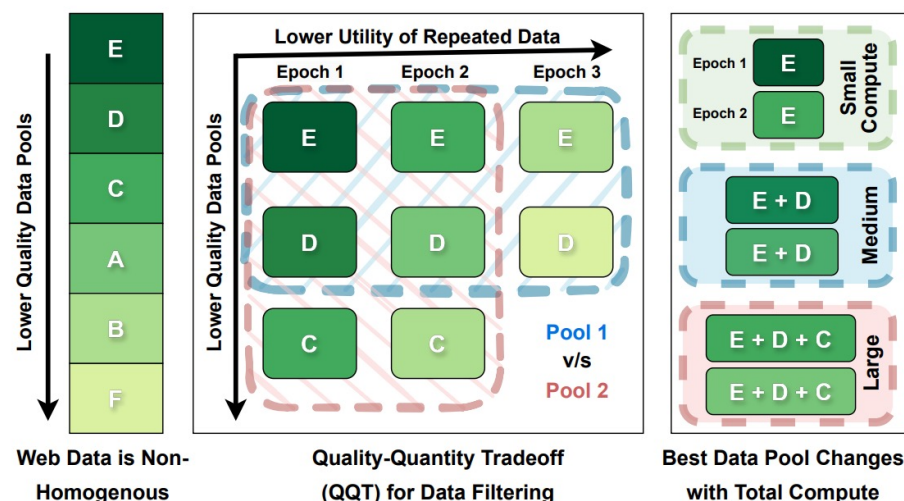
Input: Then, we will change these sentences. Each setence will be modified.
Output:

Then, we will change these sentences. Each setence will be modified.



Inverse Scaling: When Bigger Isn't Better. TMLR 2023

Limitations of Scaling Laws



- The optimal data filtering strategy depends on the compute budget:
 - small compute: cleaner and smaller datasets work best;
 - with large compute, including noisier data can be beneficial.
- Current scaling laws often assume that all web tokens are equally informative, ignoring heterogeneity in data quality.

Goyal S, Maini P, Lipton Z C, et al. Scaling Laws for Data Filtering--Data Curation cannot be Compute Agnostic. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024: 22702-22711.

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (short answer): Write down the relationship between N , C , D mentioned in this lecture.
 - Q2 (T/F): Each matrix-vector calculation Ax requires computation complexity linear in the number of columns of the matrix A
 - Q3 (multi-choice): which of the following is a property of scaling laws: (A) Kaplan's scaling law says pre-training cross-entropy loss decreases faster if increase the model size than the number of tokens; (B) inverse scaling can happen due to data containing undesirable patterns; (C) the IsoFLOPs curves in Chinchilla scaling law is based on the optimal model sizes.
 - Answers: $C=6ND$; N/A (due to ambiguity of the question); (AB)